

UNIT - I

Introduction: Purpose of testing, Dichotomies, model for testing, consequences of bugs, taxonomy of bugs.

Flow graphs and Path testing: Basics concepts of path testing, predicates, path predicates and achievable paths, path sensitizing, path instrumentation, application of path testing.

1.1 What is testing?

Testing is the process of evaluating a system or system components by manual or automated means to verify that it satisfies specified requirements.

It usually takes a lot of time and effort during software development, and this effort is wasted if the tests do not find the existing errors. Even after testing carefully, some mistakes can still appear, because no code is perfect. The best way to reduce these errors is to create good test plans and test as many parts of the software as possible.

1.2 PURPOSE OF TESTING

Testing takes at least half of the time and work required to produce a software program.

- MYTH: Good programmers write code without any bugs, but this is not true.
- History says that even well written programs still have 1-3 bugs per hundred statements.

i) Productivity and Quality in software:

- In manufacturing, every step of making a product is checked for quality and tested.
- If errors are discovered at any stage, the product is either discarded or cycled back for rework and correction.
- Productivity is measured by the sum of the cost of the material, the rework, and the wasted components, quality assurance and testing.
- There is a trade-off between quality assurance cost and manufacturing cost: If sufficient time is not spent in quality checks, many products will fail and cost more. If too many checks are done, examination becomes expensive.
- For physical products, testing costs can be very low (like 2%) or extremely high (up to 80%) for safety-critical items like aircraft or nuclear reactors. In software, the cost of making a copy is almost zero.
- The major cost in software comes from bugs—finding them, fixing them, designing tests, and running tests. Because of this, software quality and productivity are closely linked.

ii) Goals for testing:

The main goal of testing is to make sure software works correctly by preventing bugs and finding problems early. Good testing not only detects errors but also provides clear information that helps developers fix them quickly. Preventing bugs is always better than fixing them later because it saves time, avoids retesting, reduces effort, and keeps the project on schedule. By designing good tests and thinking carefully during test creation, issues can be caught at any stage of development, improving the overall quality of the software.

iii) Phases in a Tester's Mental Life:

Phases in a tester's mental life can be categorized into the following 5 phases:

1. Phase 0: Debugging Oriented (Before 1956)

In this phase, people did not see any difference between testing and debugging. Both activities were treated as the same checking the program and fixing errors. Testing was not recognized as a separate process in software development during these early years.

2. Phase 1: Demonstration Oriented (1957–1978)

During this time, testing was used mainly to prove that the software works correctly. However, this approach failed because the more you test a program, the more bugs you are likely to find. So, testing could never truly prove that the software was perfect.

3. Phase 2: Destruction Oriented (1979–1982)

Here, the goal of testing was the opposite to show that the software does not work. This also failed because testers would keep finding one bug after another, and fixing one bug often created new ones. This meant the product would never be ready for release.

4. Phase 3: Evaluation Oriented (1983–1987)

In this phase, testing focused on reducing the risk of software failure to an acceptable level. The idea was not to prove anything but to improve the product by finding bugs and fixing them. The software was released when developers felt confident that the remaining risk was very low.

5. Phase 4: Prevention Oriented (1988–2000)

This phase highlighted preventing bugs before they occur. The focus shifted to writing code that is easy to test, because testable code usually has fewer bugs. The main goal was to identify the best testing techniques and make testing easier and more effective.

iv) Test Design:

Test Design is the process of creating tests to check whether software works correctly. Good tests should be designed carefully and verified before using them on the real software.

There are many other methods, apart from testing, that help make software better.

- **Inspection Methods:** Methods like walkthroughs, reading code, and reviews help find errors. They can catch bugs that normal testing might miss.
- **Design Style:** If the software is designed in a clear and simple way, many bugs can be avoided from the beginning.
- **Static Analysis Methods:** These methods check the code without running it. Today, compilers automatically do many of these checks.
- **Languages:** Some programming languages reduce certain types of mistakes. Programmers find new bugs while using new languages.
- **Development Methodologies & Environment:** Good development processes and tools help prevent bugs early and improve the overall quality of the software.

1.3 DICHOTOMIES

Dichotomies in software testing mean dividing concepts into two opposite categories to make them easier to understand and analyze.

i) Testing Versus Debugging:

Testing and debugging are often confused, but they have different purposes. The purpose of testing is to check the program and show that there are errors in it. Debugging, on the other hand, comes after testing and focuses on finding the exact cause of those errors and fixing them. In simple terms, testing finds the problems, and debugging solves them.

Testing	Debugging
Testing starts with known conditions and follows fixed steps with expected results.	Debugging starts with unknown conditions, and the final outcome is uncertain.
Testing can be planned, designed, and scheduled.	Debugging cannot be strictly planned.
Testing shows that an error exists or that the software seems correct.	Debugging is the process of finding the exact cause of the error.
Testing exposes a programmer's mistakes.	Debugging helps the programmer explain and fix those mistakes.
Testing can be done without deep design knowledge.	Debugging needs strong understanding of the software design.
Testing can be done by an outsider (tester).	Debugging must be done by an insider (developer).
Many testing tasks can be automated.	Automated debugging is not fully possible yet.

ii) Function Versus Structure:

In **functional testing**, the program is treated like a *black box*. We give inputs and check if the outputs are correct. We do not look at how the program is built. This is from the user's point of view, focusing only on what the software does. In **structural testing**, checks the inside of the program, including the code, logic, and design. Both functional and structural tests are important, and each has limitations. Functional testing could find all bugs but would take too much time. Structural testing is limited and faster but cannot find every error.

iii) Designer Versus Tester:

- ✓ **Designer:** Creates the software, writes the code, and ensures it meets specifications. Their focus is on building the system.
- ✓ **Tester:** Evaluates the software to find bugs and verify correctness. Their focus is on breaking or validating the system to improve quality.

iv) Modularity Versus Efficiency:

- ✓ **Modularity:** Focuses on designing software in separate, independent modules. It makes code easier to understand, maintain, and test.
- ✓ **Efficiency:** Focuses on making software fast and resource-friendly, even if the design becomes more complex.

v) Small Versus Large:

Small programs or modules that are easier to understand, test, and debug. Large programs or systems which are more complex, harder to test thoroughly. Big systems are more complex than small ones.

vi) Builder Versus Buyer:

Most software is made and used by the same organization, but this is not good. If the builder and the user are the same, it becomes hard to know who is responsible when something goes wrong. If there is no separation between builder and buyer, there can be no accountability.

The different roles / users in a system include:

- **Builder:** Develops the software, handling design, coding, and implementation.
- **Buyer:** Acquires the software and checks if it meets their needs.
- **User:** Uses the software for real tasks, focusing on functionality and ease of use.
- **Tester:** Finds bugs and verifies correctness, ensuring quality and reliability.
- **Operator:** Runs and maintains the software in production, ensuring smooth operation and stability.

1.4 MODEL FOR TESTING

All the systems need to be tested. The typical system is that which allows exploration of all testing aspects without any complications. Given below in figure 1.1 is a model of testing:

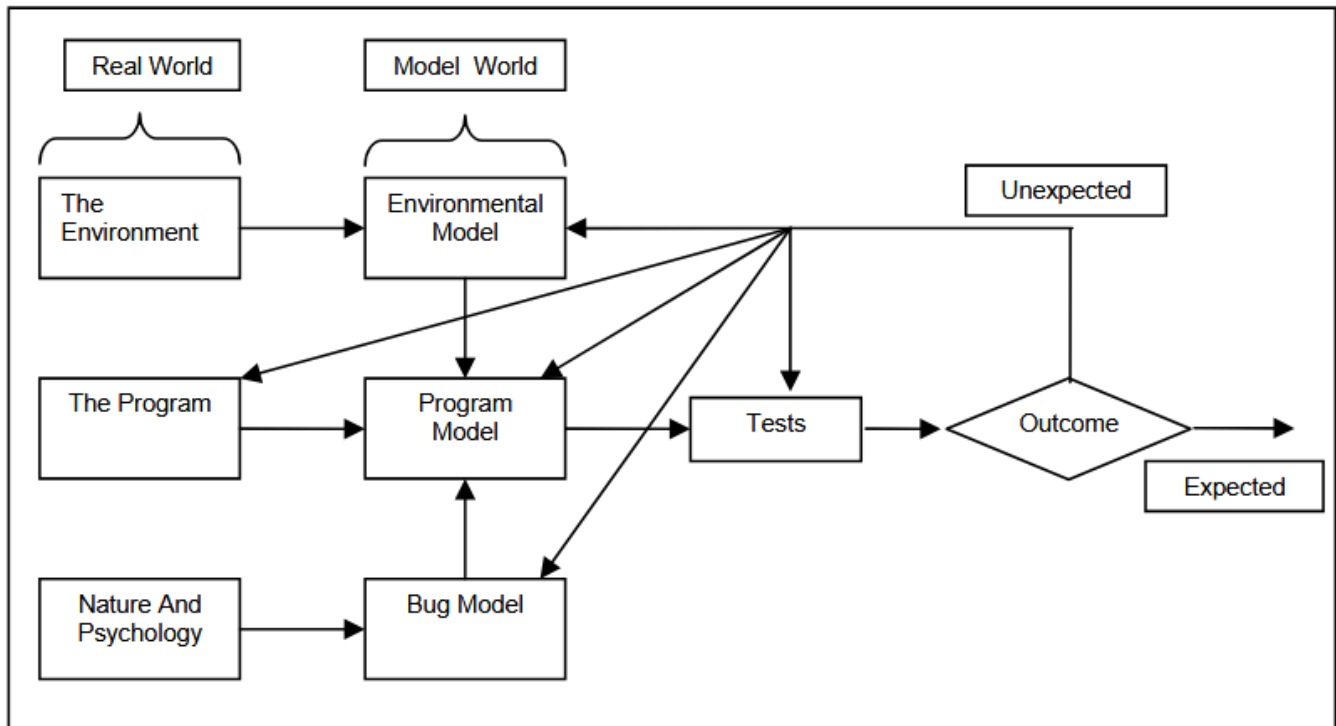


Figure 1.1: A Model for Testing

Figure 1.1 shows the testing process. It starts with a program running in an environment, like an operating system or another program. To test it, we use three models: a model of the environment, a model of the program, and a model of likely bugs. Using these models, we create tests and run them. The results may be expected or unexpected. If the results are unexpected, we may need to change the test, the models, the program, or changing the environment is rarely needed.

i) Environment: A program's environment includes the hardware and the software required for making it run. It also includes all the programs interacting with it and used to create this program under test e.g. operating system, loader, linker, compiler, etc. A testing environment has a test plan, which lists the tests to be done, and test cases, which explain how each part of the program will be tested. Creation of a test environment reduces the risk of errors occurring when the software is used.

ii) Program: Many programs are too complex to understand, so we simplify them for testing by ignoring some details. If the simplified model doesn't explain unexpected behavior, we can add more details to the model. And if that fails, we may have to modify the program.

iii) Bugs: A bug is an error in software that causes it to behave unexpectedly or incorrectly. Bugs can occur in code, design, requirements, or even in how the software interacts with hardware or users. These are listed below:

- **Benign Bug Hypothesis:** Only simple bugs are predictable and easy to find; subtle bugs are unpredictable and have no clear pattern.
- **Bug Locality Hypothesis:** A bug in one module can affect other modules, it is not always true.
- **Control Bug Dominance:** Believing most bugs are in control structures like if or switch.
- **Code / Data Separation:** Bugs do not always respect the boundary between code and data.
- **Lingua Salvatore Est.:** Programming language does not automatically eliminate bugs.
- **Corrections Abide:** Fixing a bug once does not guarantee it won't appear again.
- **Silver Bullets:** There is no single tool, method, or language that can prevent all bugs.
- **Sadism effect:** Bugs cannot be found by intuition or cleverness alone; systematic testing methods are needed.

iv) Tests: Tests follow a formal process. we prepare inputs, predict the outcomes, tests should be documented, commands need to be executed, and results are to be observed. But mistakes can happen at any of these steps.

v) Testing Levels: Software is tested at different levels to ensure each part works correctly, both individually and together. Each type has a different purpose.

- **Unit / Component Testing:** A unit is the smallest part of a program that can be tested. It is usually written by a programmer and has only a few lines of code. Unit testing checks whether this small part meets its requirements and follows the intended design. Any problems found here are called unit bugs.
- **Component Testing:** A component is made by combining one or more units. Even a single unit can be considered a component, depending on the design. Component testing checks whether this larger combined piece works properly. Bugs found here are called component bugs.

- **Integration Testing:** Integration means joining components to form larger subsystems. Integration testing checks whether these connected components work correctly together.
- **System Testing:** A System is a big component. System testing checks the whole system to find bugs that don't belong to any single part. It includes testing for performance, security, accountability, recovery and reliability.

v) Role of Models:

Testing involves creating, choosing, checking, and updating different models. We can do this well only if we have many useful models that explain how the program works.

1.5 CONSEQUENCES OF BUGS

i) Importance of bugs:

The importance of bugs depends on frequency, correction cost, installation cost, and consequences.

1. **Frequency:** It is important to understand how often a bug appears and which bugs occur the most frequently.
2. **Correction Cost:** It is important to know how much it will cost to fix a bug once it is found. This cost includes finding the bug and correcting it. The cost becomes much higher if the bug is discovered late in the software process, and fixing bugs in large programs is even more expensive.
3. **Installation Cost:** Installation cost depends on the number of installations. It is low for one user but much higher for large distributed systems.
4. **Consequences:** The impact of bugs can be measured by the average compensation paid to the people affected by them.

The metric for the measurement of bug importance is:

$$\text{Importance} = (\$) = \text{Frequency} * (\text{Correction cost} + \text{Installation cost} + \text{Consequential cost})$$

ii) How Bugs Affect Us – Consequences / Consequences of bugs:

The effects of bugs can be small or very serious. We should measure their impact on people, not on machines, because software is made for humans to use.

Some consequences of a bug on a scale of one to ten are:

1. **Mild:** if a word is misspelled on the screen or the text is not properly aligned, it feels unpleasant or “ugly” to the user. These errors affect the appearance of the output, so they offend our sense of aesthetics.
2. **Moderate:** Outputs are misleading or redundant. and the bug slows down the system.
3. **Annoying:** The system behaves in a way that feels disrespectful or irritating.
4. **Disturbing:** The system refuses valid actions that should work normally. Example: ATM won't give cash even though your account is fine, or your valid card is shown as invalid.
5. **Serious:** The system forgets transactions completely, including the proof that they happened.
6. **Very Serious:** The system performs wrong transactions.
7. **Extreme:** The bugs affect many users, happen frequently, and occur in normal situations, not just rare cases.
8. **Intolerable:** The database becomes permanently corrupted, and it's very hard to detect or fix.

9. **Catastrophic:** The system completely fails, and shutting it down becomes unavoidable.
10. **Infectious:** the system damages other systems or environments.

iii) Flexible severity rather than absolutes:

Quality of a program can be measured from the combination of number of bugs and their severity. Bugs and their effects play a significant role. During testing, software quality improves from almost zero to a reliable for release.

The factors involved in bug severity are:

1. **Correction Cost:** Fixing a bug doesn't always match its seriousness. Big bugs can be easy to fix, while small bugs may take a long time.
2. **Context and Application Dependency:** The seriousness of a bug depends on the type of software and where it is used.
3. **Creating Culture Dependency:** The importance of a bug depends on the software creators and their priorities.
4. **User Culture Dependency:** Severity also depends on user culture. Beginners may get upset over small bugs, while experts may ignore them.
5. **The software development phase:** Severity depends on development phase. Bugs become more serious as the software gets closer to release, and older bugs are considered more severe.

1.6 TAXONOMY OF BUGS

There is no perfect way to categorize bugs. Different people may categorize the same bug in different ways because it depends on the background of the bug and how the programmer understands it. So, one bug can belong to different categories in different situations. The major categories are: (1) Requirements, Features and Functionality Bugs (2) Structural Bugs (3) Data Bugs (4) Coding Bugs (5) Interface, Integration and System Bugs (6) Test and Test Design Bugs.

i) Requirements, Features and Functionality Bugs:

Various categories in Requirements, Features and Functionality bugs include:

- **Requirements and Specifications Bugs:** Requirements and specifications are often unclear or incomplete, and people may misunderstand them. They can also change during design when features are added or modified. Because of this, many bugs start in the requirements stage and often remain in the software until the end.
- **Feature Bugs:** If there are problems in the specifications, the features will also have problems. A feature may be wrong, missing, or not needed at all. Missing features are easier to find, but wrong features can cause serious design issues. Extra features can also make the software more complex, waste resources, and even create more bugs.
- **Feature Interaction Bugs:** Even if each feature is clearly defined, problems can still occur when features interact. Features usually work in groups, and while each group is tested properly, unexpected interactions between different groups can still create bugs.

ii) Structural bugs:

Various categories in Structural bugs include:

- **Control and Sequence Bugs:** Control and sequence bugs occur when certain paths are missing, some code cannot be reached, loops are incorrect, or important steps are skipped. They can also come from bad use of GOTO statements or poorly designed switch statements.

Example:

```
i = 100;
a = 20;
If (i < a) then
    Print "i is less than a";
Else
    Goto label1:
    :
Label 2: j = i + 2;
```

The above program, the control bug happens because the code tries to go to “label 1” in the else part, but that label does not exist. So, when the else condition is true, the program does not know where to go next, causing an error.

- **Logic Bugs:** Logic bugs occur when logical statements or operators are used incorrectly. This can include missing cases, wrong order of cases, incorrect simplification, or overlapping cases. If these mistakes only affect logic, they are called processing bugs.

Example:

```
scanf("%d", &money);
if (money_in_store == 0)
{
    printf("%d money on exit\n", money_in_store);
    exit(0);
}
```

This is an example of logical bug. After ‘scanf’, we are checking for ‘money_in_store’ instead of ‘money’.

- **Processing Bugs:** Processing bugs include arithmetic, algebraic, mathematical function evaluation, algorithm selection, incorrect conversion from one data type to another and general processing. Other common problems are ignoring overflow, not handling positive and negative zero correctly, and using comparison operators like greater than (>) or less than (<) wrongly. For example, dividing a number by zero (b = 10 / 0) causes a serious processing error because such a calculation is not allowed.
- **Initialization Bugs:** Initialization bugs occur when variables are not properly prepared before use. Some unnecessary initializations may slow down the program. Common mistakes include forgetting to initialize a variable, assuming it already has a value, using the wrong type, or using an incorrect format.

Example:

```
int a;
char b = 'x';
a = b;
```


The program above gives an initialization error as we are trying to assign the value of a character type variable to an integer variable.

iii) Data bugs:

Data bugs happen when there are mistakes in how data is defined, its format, the number of data items, or their initial values. These bugs are as common as code bugs, but they are often overlooked or ignored.

- **Third Law - Code migrates to data:** This law highlights that bugs in the code. It also shows that code bugs are only part of the problem, and data-related bugs are equally important.
- **Dynamic Data Vs Static data:**
 - ✓ **Dynamic data bugs:** These happen when leftover data remains in a shared resource. They can be handled in three ways: (1) user cleans up after use, (2) system/resource manager cleans up, or (3) no cleanup.
 - ✓ **Static data:** These are fixed values like numbers, strings, or bit patterns in code or databases. Bugs in static data are usually detected and fixed during compile time.
- **Information, parameter, and control:** Static or dynamic data can be used in three ways, or a combination of them: as a parameter, to control the program, or to store information.
- **Content, Structure and Attributes:**
 - ✓ **Contents:** These are the actual bits, numbers, or characters in a data object. They have no meaning until interpreted by software or hardware.
 - ✓ **Structure:** This is the size, shape, and organization of the data, including memory used.
 - ✓ **Attributes:** These are the meanings or types assigned to the data, like integer, string, or subroutine.

iv) Coding bugs:

Coding bugs can lead to other bugs. Syntax errors are usually detected by the compiler, including undeclared variables, missing routines, or unused code. These errors must be fixed before testing, because if there are many syntax errors, the program may also have many logic and coding bugs.

v) Interface, integration, and system bugs:

Various categories of bugs in Interface, Integration, and System Bugs are:

- **External Interfaces:** These allow a program to interact with the outside world, such as devices, sensors, printers, or communication lines. They should be reliable and follow proper communication protocols.
- **Internal Interfaces:** Internal interfaces are like external interfaces, but the internal environment is more controlled. Unlike external interfaces, which must adapt to fixed conditions, internal interfaces connect components and can be adjusted. They may have the same problems as external interfaces, along with issues from their implementation.
- **Hardware Architecture:** Hardware-related bugs happen when software misunderstands the hardware. Examples include I/O errors, wrong data formats, or wrong device handling.

- **Operating System Bugs:** Operating system (OS) bugs happen when programmers misunderstand the operating system, combining hardware and interface issues.
- **Software Architecture:** Software architecture bugs, also called “interactive” bugs, these may not appear during unit or integration testing. They often appear only under heavy system load and are very hard to detect.
- **Control and Sequence Bugs (Systems Level):** Control and sequence bugs happen when timing or order of events is wrong. Examples include ignoring timing, assuming events happen in a certain order, starting a process too early, waiting for impossible conditions.
- **Integration Bugs:** Integration bugs appear when tested components are combined. They usually happen because components don’t work well together or are incompatible.
- **System Bugs:** System bugs are caused by interactions between many components, like programs, data, hardware, and the OS, rather than by a single component.

FLOW GRAPHS, PATH TESTING AND TRANSACTION FLOW TESTING

1.7 BASICS OF PATH TESTING

i) Motivation and Assumption/ Path Testing: Path testing focuses on selecting specific paths through a program to check its behavior. By choosing the right paths, every part of the code can be executed at least once. It is an old structural testing method, mainly used for unit testing by programmers. Path testing requires complete knowledge of the program’s structure, but its effectiveness decreases as the program size increases.

ii) Control Flow Graphs: The control flow-graph (or flow-graph) is a graphical representation of a program’s control structure. It is similar to the flowchart. It uses the elements as shown in Figure 4.1: process blocks, decisions and junctions.

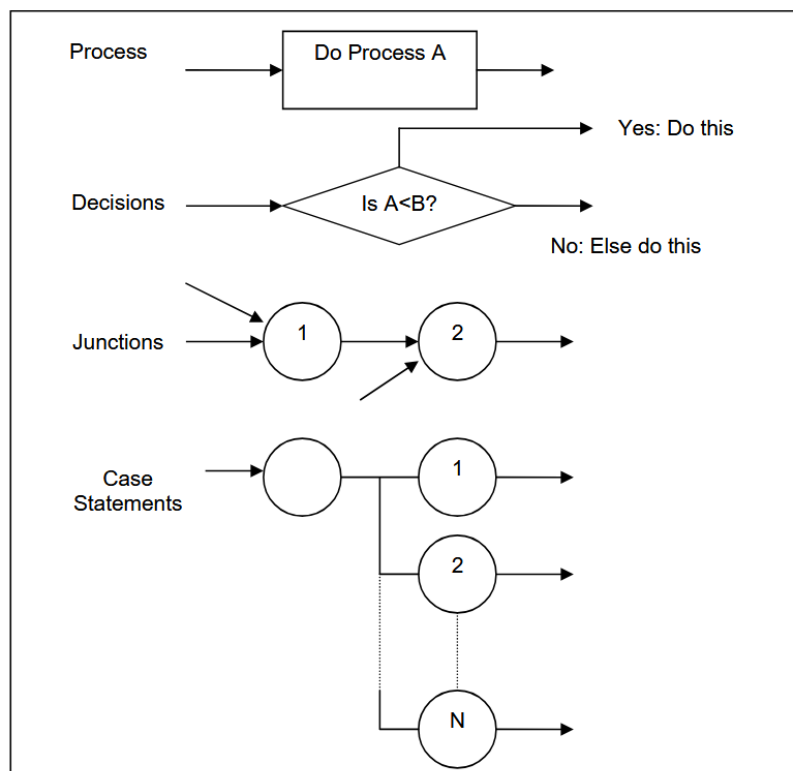


Figure 4.1: Flow-graph Elements

- **Process Block:** A **process block** is a sequence of statements without any decisions or junctions. If one statement in the block is executed, all statements are executed. It can contain a single statement or hundreds. Each process block has **one entry and one exit** and may include statements, instructions, macros, functions, or a combination of these.
- **Decisions and Case Statements:** A decision is a point in a program where the control can go in different directions. Examples include conditional branches. A case statement is a decision with multiple branches.
- **Junctions:** A junction is a point in the program where the control flow can merge. Examples include a jump target or a label used as the destination of a GOTO statement.

iii) Path Testing:

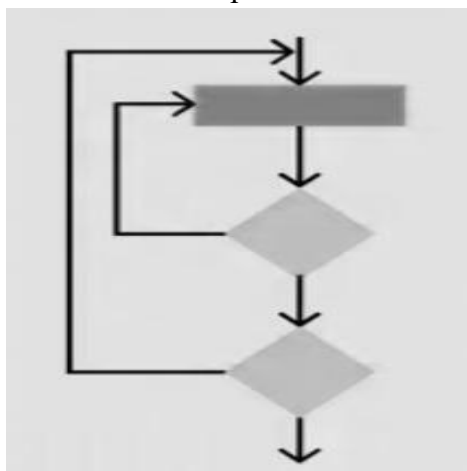
- **Paths, Nodes and Links:** A path is a sequence of statements, starting from an entry point and ending at another point or the exit. A single process may be visited more than once in a path. Path consists of segments, and the smallest segment is called a **link**, which is a single process between two nodes. A **path segment** is a group of links that come one after another. The **length of a path** is measured by counting how many links or nodes it goes through.
- **Fundamental Path Selection Criteria:** Fundamental Path Selection Criteria are guidelines used to decide which program paths should be tested to get good test.

Complete testing means:

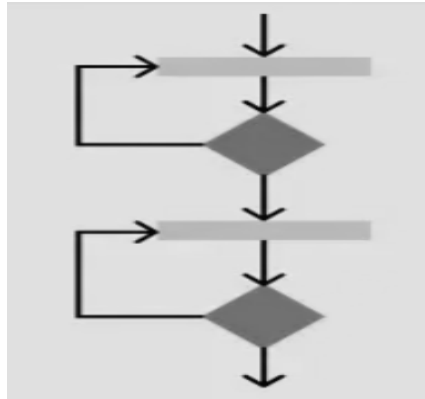
1. Test every path from entry to exit.
 2. Test every statement at least once.
 3. Test every branch or case in all directions at least once.
- **Path Testing Criteria:** Path Testing Criteria provide guidelines for selecting test cases to cover different paths in a program. we have discussed three testing strategies.
 - ✓ **Path Testing (P00):** Test all possible paths from entry to exit. This gives 100% path coverage. It is the strongest path-testing criterion, but usually impossible to achieve.
 - ✓ **Statement Testing (P1):** Test every statement at least once. This gives 100% statement coverage called C1. This is the simplest criterion in path testing.
 - ✓ **Branch Testing (P2):** Every branch is executed at least once. This gives 100% branch coverage, called C2.

iv) Loops: There are three kinds of loops: nested, concatenated and horrible loops.

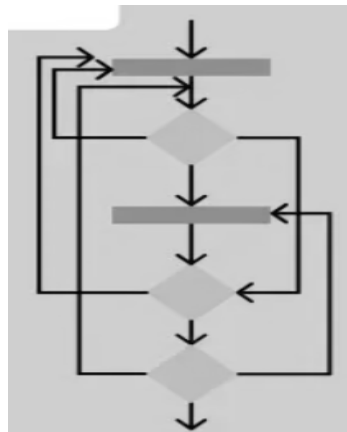
- **Nested Loops:** Nested Loops are loops placed inside another loop, so that the inner loop executes completely for each iteration of the outer loop.



- **Concatenated Loops:** Concatenated loops are loops that appear one after another on the same execution path. Once the first loop finishes, control moves to the next loop.



- **Horrible Loops:** Horrible loops are un-structured loops that include jumps into or out of loops using statements like GOTO, hidden loops, or cross-linked loops. Horrible Loops are loops that are difficult to test because of their complex structure or behavior.



v) Variations: Variations refer to different methods used in path-based testing to handle complexity and improve effectiveness. They mainly fall into two classes:

1. Strategies between C2 (branch coverage) and full path testing.
2. Strategies weaker than C1 (statement coverage) or C2.

1.8 PREDICATES, PATH PREDICATES AND ACHIEVABLE PATHS

i) Predicates: A predicate is a Boolean condition (true or false) used in a decision statement. The value of this condition decides which path the program follows. This expression is called a **predicate**. Each program path is defined by a sequence of true/false results from these predicates. The predicate that represents a particular path is called a **path predicate**.

ii) Predicate Expressions: A **predicate expression** is a condition that has been simplified by rewriting all earlier operations in terms of the original input variables. The main purpose of a predicate expression is to find exactly which input values will make the condition on a chosen path evaluate to **true or false**.

Example: function example(x):

$$y = x + 1$$

$$z = y * 2$$

```
if z == 24:  
    // Final check
```

Predicate Interpretation:

1. The condition is $z == 24$.
 2. We substitute $z = y * 2$ into the condition: $(y * 2) == 24$.
 3. We substitute $y = x + 1$ into the condition: $((x + 1) * 2) == 24$.
 4. *The final predicate expression is $2x + 2 == 24$.* This expression depends only on the input x .
- **Independence of Variables and Predicates:** Path predicates take TRUE or FALSE values from the input variables, either directly or indirectly. If a variable's value does not change during program execution, it is called **process independent**. If its value can change because of processing, it is **process dependent**. Similarly, a predicate is **process independent** if its truth value does not change during processing, and **process dependent** if its truth value can change due to processing.
 - **Correlation of Variables and Predicates:** Variables are said to be **correlated** if their values are linked. If each variable can take any value without affecting the other, they are called **uncorrelated**.

iii) Predicate Coverage:

Compound predicates are conditions formed by combining two or more simple conditions using logical operators like **AND** or **OR**. The decision taken depends on the final TRUE or FALSE value of the whole expression. Predicate coverage means testing all possible truth-value combinations of the conditions involved on a selected path.

iv) Testing Blindness: Testing Blindness occurs when a test follows the correct program path, but not for the correct reason. This means a bug exists, but the test does not detect it. There are three types of Testing Blindness:

- **Assignment Blindness:** Assignment Blindness occurs when a wrong assignment does not show a bug because the chosen test value works for both the correct and incorrect code.

For Example:

Correct code:

```
x = y + 1;  
if (x > 10)
```

Buggy code:

```
x = y;  
if (x > 10)
```

If we test with $y = 11$, both versions make $x > 10$ true, so the program seems correct. But with $y = 10$, the correct code gives $x = 11$ (true) and the buggy code gives $x = 10$ (false), revealing the bug.

- **Equality Blindness:** Equality Blindness happens when a bug is missed because a value set earlier satisfies both the correct and the wrong condition, so the program follows the same path and the error is hidden.

For Example:

Correct code:

```
if (x == 10)
```

Buggy code:

```
if (x >= 10)
```

If a previous step sets $x = 10$, both conditions evaluate to **true**, so the program follows the same path and the bug is not detected. But if $x = 12$, the correct condition becomes **false** while the buggy one is **true**, exposing the error.

- **Self-Blindness:** Self-blindness occurs when a wrong condition in the program behaves exactly like the correct condition. Because both give the same result, testing cannot detect the bug.

For Example:

Correct code:

if (x > 5)

Buggy code:

if (2*x > 10)

Both conditions mean the same thing, so the program follows the same path every time.

1.9 PATH SENSITIZING

Path Sensitizing is the process of finding input values that will make a program follow a specific path during testing. It confirms that the selected path is actually **achievable** and helps in designing effective test cases.

The process for path-based testing (Achievable and unachievable paths):

1. Select paths to satisfy test coverage criteria (C1 and C2).
2. Extract the program's control flow graph.
3. For each path, rewrite the predicates along the path in terms of the input vector.
4. Combine all predicates along the path into a single Boolean expression, like

$$(A + BC)(D + E) (FGH)(IJ)(K)(L).$$
5. Expand the expression into sum-of-products form, e.g.,

$$ADFGHIJKL + AEF GHIJKL + BCDF GHIJKL + BCEFGHIJKL.$$
6. Each product term represents conditions (inequalities) for input values.
7. Solve one set of conditions to get input values for the chosen path.
8. If a solution exists, the path is achievable.
9. If no solution exists, the path is unachievable.
10. The process of finding input values from the path predicate expression is called **path sensitization**.

Heuristic Steps for Sensitizing Paths:

1. Selecting paths randomly, first choose paths that are easier to sensitize.
2. Identify all variables that affect decisions.
3. Classify predicates as dependent or independent.
4. Start selecting paths using independent and uncorrelated predicates.
5. If coverage isn't complete, add paths with correlated predicates.
6. If coverage is still incomplete, include paths with dependent predicates.
7. Finally, use paths with correlated and dependent predicates.

1.10 PATH INSTRUMENTATION

Software testing instrumentation involves adding code or tools to a program to monitor its behavior, performance, and code coverage during execution. The main types of instrumentation methods focus on tracking execution paths and gathering data. The types of instrumentation methods include:

1. Interpretive Trace Program: These programs run every statement in order and record lots of details, like intermediate values and which statements were executed. While this gives complete information, the huge amount of output can be overwhelming and makes it hard to confirm the correct path.

2. Traversal Marker or Link Marker: A simple and effective instrumentation is called a traversal marker or link marker. Each link in the program is assigned a letter, which is recorded when the link is executed. The sequence of letters from start to finish should match the expected path if there are no bugs. However, a single link marker may not be sufficient because links can be affected due to bugs.

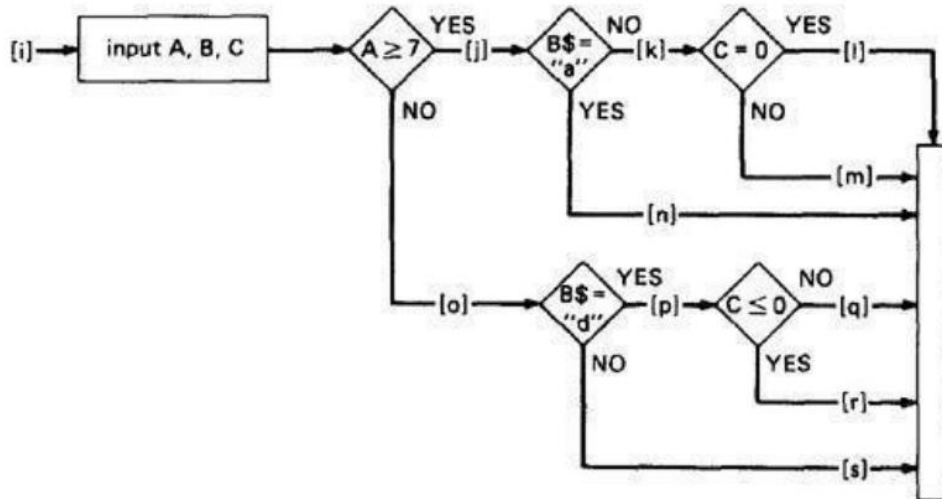
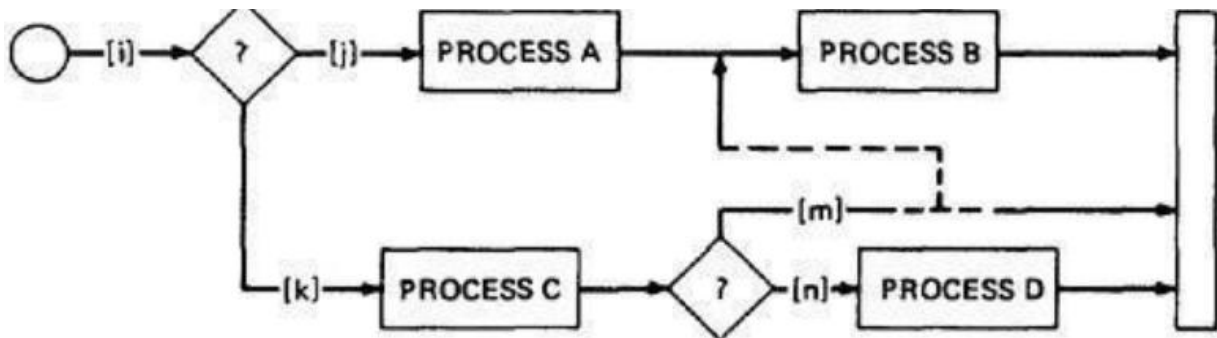


Figure 2.12: Single Link Marker Instrumentation

Why single link markers aren't enough: A single link marker may fail because bugs can alter the links, preventing accurate tracking of the path.



We planned to follow the **ikm** path, but a faulty GOTO in the **m** link takes us to process B instead. If coincidental correctness occurs, the results may look correct, hiding the bug.

3. Two Link Marker Method: The solution to the single link marker problem is to use **two markers per link**: one at the start and one at the end. This way, the markers specify the path and confirm that each link was executed from beginning to end.

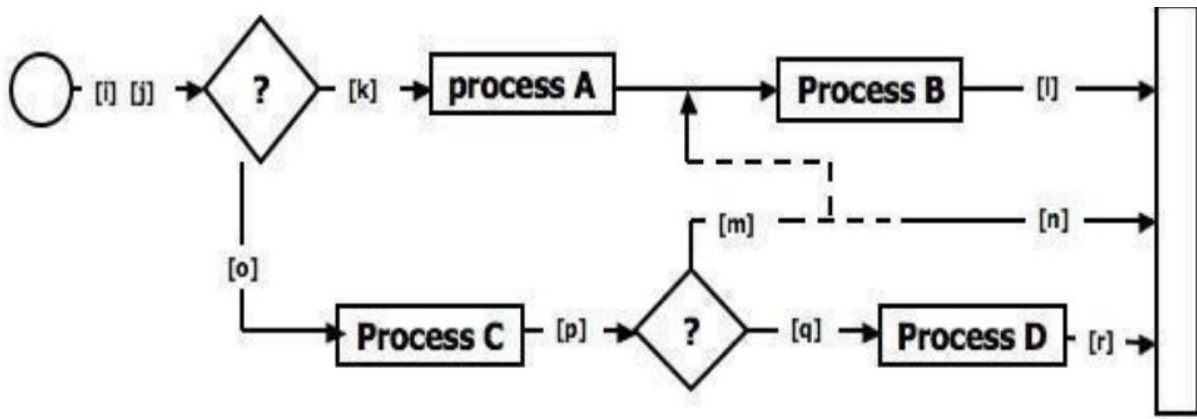


Figure 2.14: Double Link Marker Instrumentation

4. Link Counter: A simple way to track program paths is using **counters**. Each time a link is executed, the counter is increased to check the path length. Like link markers, **two counters per link** may be needed to confirm the link runs from start to end.

1.11 INSTRUMENTATION AND APPLICATION OF PATH TESTING

Instrumentation: Refer path instrumentation

Application of Path Testing:

Path testing is a method of testing a program by focusing on specific paths through its code. Its application includes:

1. **Select Paths:** Identify paths from the control flow graph of the program.
2. **Path Sensitization:** Determine input values that will make the program follow the chosen path.
3. **Execute Tests:** Run the program with these inputs and use instrumentation (like markers or counters) to track the path.
4. **Compare Outcomes:** Verify that the actual outputs and path traversed match the expected results.
5. **Coverage Measurement:** Measure how much of the code is tested using metrics like statement coverage (C1) or branch coverage (C2).